Automated Canny Edge Detection

An Engineering Project in Community Service

Project Report

Submitted by

23BAI10525 - JAYESH WAGH

Computer Vision (CSE3010)

in partial fulfillment of the requirements for the degree of

Bachelor of Technology

VIT Bhopal University
Bhopal
Madhya Pradhesh

March 2026

Project Report: Automated Canny Edge Detection System

Student Name: Jayesh Wagh

Reg. No: 23BAI10525

Course: Computer Vision (CSE3010)

Project Component: Bring Your Own Project (BYOP)

---

1. The Problem Chosen

The problem addressed in this project is the efficient extraction of structural information (edges) from digital images. In computer vision, raw images contain vast amounts of data, much of which is redundant. For tasks like object recognition or motion tracking, we need to reduce the image to its essential "sketch" or boundaries. This project automates that extraction process for bulk image datasets.

2. Why It Matters

Edge detection is a fundamental building block in image processing. It matters because:

- **Data Reduction:** It significantly reduces the amount of data to be processed while preserving the structural properties of objects.
- **Feature Extraction:** It serves as a precursor to more advanced AI tasks like face detection or autonomous vehicle lane tracking.
- **Automation:** Manually editing images for edge extraction is time-consuming; a programmatic approach allows for high-speed batch processing of thousands of images.

3. My Approach to Solving It

I developed a Python-based automation script using the **OpenCV** library. My approach involved:

- **Preprocessing:** Converting images to grayscale to focus on intensity rather than color, and applying a Gaussian Blur to ensure the detector doesn't get "confused" by background noise.
- **Algorithm Implementation:** Using the Canny Edge Detection algorithm, which is known for its low error rate and good localization.
- **Batch Processing Pipeline:** I built a loop that automatically scans an input folder, processes every compatible image file, and saves the result in an output folder without requiring manual intervention for each file.

4. Key Decisions Made

- **Gaussian Blur Kernel Size:** I chose a 5×5 kernel. This was a balance between removing enough noise to get clean edges and keeping the image sharp enough to not lose fine details.
- **Dual Thresholding:** I set the low threshold to 50 and the high threshold to 150. This "Hysteresis" approach ensures that only strong edges (or weak edges connected to strong ones) are kept, eliminating "ghost" edges.
- **Relative Pathing:** I decided to use relative directory paths (input/ and output/) instead of hardcoded computer paths. This ensures the project is **executable** by any evaluator on any machine.

5. Challenges Faced

- **The "Path Not Found" Error:** Initially, I used absolute paths from my local drive (G:). When testing the script, I realized this would cause the code to fail on the evaluator's computer. I solved this by implementing os.path.join and relative directories.
- **Noise Sensitivity:** Some high-resolution images had too much texture, resulting in "noisy" edge maps. I addressed this by fine-tuning the Gaussian Blur parameters.
- **File Compatibility:** Handling different file extensions (.jpg, .png, .webp) required implementing a logic check to ensure the script didn't crash when encountering non-image files in the folder.

6. What I Learned

Through this project, I gained a deep understanding of:

- **Computer Vision Fundamentals:** I learned how mathematical gradients are used to find edges in an image.
- **Software Engineering Practices:** I learned the importance of creating "portable" code that can run via the command line, which is a key requirement for professional software delivery.
- **Documentation:** I realized that clear setup instructions in a README are just as important as the code itself for a successful project submission.